

UNIT-6

Advances in Databases

 **Emerging Databases:** Emerging databases refer to modern database systems designed to address new challenges and demands in data management.

❖ **Active databases:**

- Active databases are databases which consists of triggers. An **active database** is one that automatically responds to certain conditions or events through triggers and rules. It enables event-driven actions within the database.
- An active database uses triggers that activate before executing a statement. If active, the condition is checked, and the action part runs only if the condition is true.
- These databases are very difficult to be maintained because of the complexity that arises in understanding the effect of these triggers.

Example (SQL Trigger):

sql

 Copy code

```
CREATE TRIGGER update_salary
AFTER UPDATE ON employees
FOR EACH ROW
WHEN (NEW.salary > 10000)
BEGIN
    INSERT INTO audit_log VALUES ('Salary updated', NOW());
END;
```

Here, an audit log is automatically updated when an employee's salary exceeds a specific limit.

- **Advantages:**

1. **Automation of Tasks:** Active databases automate repetitive tasks, such as enforcing constraints, auditing, or sending notifications, without requiring manual intervention.

2. **Enforcement of Business Rules: Triggers** help enforce business rules (e.g., checking credit limits, validating data) consistently, ensuring data integrity and accuracy.
3. **Real-Time Monitoring and Alerts:** Active databases can detect changes and respond instantly by sending alerts or executing actions, enabling real-time monitoring.
4. **Simplified Application Logic:** By handling events and conditions within the database, triggers reduce the need for complex application code, improving maintainability.
5. **Data Integrity Maintenance:** Triggers can ensure data consistency across tables (e.g., cascading updates or deletes), maintaining database integrity automatically.

- **Features:**

1. It possesses all the concepts of a conventional database i.e. data modelling facilities, query language etc.
2. It supports all the functions of a traditional database like data definition, data manipulation, storage management etc.
3. It supports definition and management of ECA rules.
4. It detects event occurrence.
5. It must be able to evaluate conditions and to execute actions.
6. It means that it has to implement rule execution.

- ❖ **Deductive databases:**

- Deductive database is a database that can make deductions based on rules and facts stored in the database.
- **Datalog** is the language typically used to specify facts, rules and queries in deductive databases.
- It uses the concept of logic programming for specifying the rules and the facts. **Prolog** is the popular programming language based on the logic programming.

- **Advantages:**

1. Logical Reasoning: Deductive databases can derive new information based on rules and facts stored in the database.
2. Simplifies Complex Queries: They allow users to write declarative queries (what to retrieve, not how), making it easier to work with complex relationships.
3. Knowledge Representation: Ideal for representing and managing knowledge in systems like AI and expert systems.
4. Reusable Rules: Rules can be reused for multiple queries, reducing redundancy and improving efficiency.
5. Better Decision Making: They help in decision support by deducing insights not directly stored in the database.

- **Features:**

1. Facts and Rules: Data is stored as facts (like rows in tables), and knowledge is defined through rules (logical statements).
2. Inference Engine: Uses logic to deduce new facts or relationships automatically.
3. Declarative Queries: Users specify what they need, and the system figures out how to compute it.
4. Logic Programming: Deductive databases are based on logic programming languages like Prolog or Datalog.
5. Derived Information: They can generate new data (derived facts) based on existing facts and rules.

- ❖ **Main memory databases:**

- These are also called as in-memory database. An **in-memory database** stores all its data in the **RAM (Random Access Memory)** of the computer instead of traditional hard disks. This makes data access **much faster** compared to traditional databases, even those using fast SSDs.

- **Features:**

1. **Faster Access:** Since data is stored in RAM, it can be accessed much quicker than disk-based storage.

2. **Volatile Storage:** RAM is temporary; if the server fails or shuts down, all data stored in the in-memory database will be lost because it is not saved on disk.
3. **Costly:** RAM is much **more expensive** than hard disks, so in-memory databases are costlier to operate.
4. **Best Use Cases:** In-memory databases are ideal for situations where **speed is critical** and **data loss is acceptable**, like storing temporary data (e.g., shopping carts on e-commerce websites).
- **Example:** Imagine you're shopping online. Product details (which are very important) are stored on a regular database, but your shopping cart (temporary data) can be kept in an in-memory database for quick access.

❖ **Semantic databases:**

- A semantic database stores data in a way that understands the meaning (semantics) and relationships between pieces of information. It organizes data using concepts, relationships, and rules instead of just rows and columns.
- **Features:**
 1. **Focus on Meaning:** Semantic databases store **what the data means**, not just the data itself. For example, "John works for Company X" shows a relationship between **John** and **Company X**.
 2. **Relationships Between Data:** They use **relationships** to connect data, making it easy to find related information (like a graph database).
 3. **Based on Ontologies:** Semantic databases use **ontologies** (like dictionaries) to define concepts and their relationships. For example, "a car is a vehicle."
 4. **Query Using Meaning:** Instead of asking for data with exact keywords, you can query using its **meaning**. This is helpful when dealing with complex searches.
 5. **Best Use Cases:** Semantic databases are great for applications requiring understanding of relationships, like **search engines**, knowledge graphs, or AI systems.
- **Example:**

If you ask, *"Show all employees who work in Company X's marketing*

department," a semantic database understands the **relationships** (employee → works for → company → department) and fetches the relevant results.

Complex Data types:

❖ **Semi-structured data:**

- Semi-structured data is a type of data that does not follow the strict tabular format of relational databases but still uses tags or markers to organize and separate different parts of the data. It provides some structure but remains flexible.
 1. **Self-Describing Structure:** Semi-structured data uses **tags or markers** to identify and organize information, making it somewhat structured but not fully rigid.
 2. **Flexible Organization:** Data of the same type can have **different characteristics** (e.g., two emails can have different attributes, like attachments or no attachments).
 3. **Common on the Internet:** Semi-structured data is common in **web applications**, emails, social media, and object-oriented databases.
 4. **Easier to Analyze:** While more structured than unstructured data, semi-structured data often needs **machine learning tools** to analyze and extract insights automatically.
 5. **Examples:**
 - **Emails:** Organized by sender, subject, and folders like Inbox or Spam.
 - **Social media:** Platforms like Facebook organize users, friends, and posts, but the comments (text) remain unstructured.
 - **Media Files:** Pictures or videos include **meta-tags** (location, date) but the content itself has no structure.

❖ **Features of semi-structured data models:**

- The **semi-structured data model** is a type of database model where the **data** and its **structure (schema)** are **not strictly separated**. Unlike relational databases, which use fixed rows and columns, semi-structured data is more **flexible** and allows data to have different structures depending on its purpose.
- 1. **Not fixed format:** Data in a semi-structured model does not need to follow a rigid, tabular format like relational databases. Different records can have different fields. **Example:** An email may have an **attachment** field, while another does not.
- 2. **Uses tags and markers:** Semi-structured data uses tags (HTML, XML), markers, or labels to describe and separate different parts of the data. These tags help define the structure of the data without enforcing a strict schema. **Example:** In an **XML** document, `<name>John</name>` tags the name of a person.
- 3. **Allow missing/Extra data:** : Semi-structured data allows different records to have missing or additional fields. **Example:** A contact record might have a **phone number** in one entry, but not in another.
- 4. **Hierarchical data:** Data can be nested or organized into a hierarchy. Semi-structured data often has sub-data or child elements within the main data, making it easy to represent complex relationships. **Example:** A blog post has **comments** nested under it, like:
 - **Post:** "Hello World"
 - **Comment:** "Great post!"
- 5. **Flexible for different applications:** Semi-structured data is adaptable to various use cases and can store diverse types of data, making it useful for modern web-based and data-driven applications. **Example:** A **Facebook post** can include **text**, **images**, or **videos**, all together.

Nested data types:

❖ JSON:

- **JSON** stands for **JavaScript Object Notation**.
- It is an **open standard data-interchange format** used to store and exchange data.
- JSON is **lightweight**, meaning it takes up less space and is faster to transmit. It is also **self-describing**, meaning the data is easy to understand because it uses a simple, readable structure.
- JSON originated from JavaScript but is now used widely across many programming languages because it is **language-independent**. This means you can use JSON to exchange data between different systems and programming languages.
- JSON supports **two main data structures: arrays and objects**, which are used to organize data.
 - **JSON objects:** In JSON, objects refer to dictionaries, which are enclosed in curly brackets, i.e., { }. These objects are written in key/value pairs, where the key has to be a string and values have to be a valid JSON data type such as string, number, object, Boolean or null.

For example:

```
{"name": "Jack", "employeeid": 001, "present": false}
```

- **JSON Arrays:** In JSON, arrays can be understood as a list of objects, which are mainly enclosed in square brackets []. An array value can be a string, number, object, array, Boolean or null.

json

```
{  
  "fruits": ["Apple", "Banana", "Orange", "Grapes"]  
}
```

❖ Encoding and decoding of JSON object in java:

- Encoding refers to the process of converting a Java object into a JSON string.
- Decoding refers to the process of converting a JSON string back into a Java object.
- To achieve encoding and decoding of JSON in Java, we can use the Gson library.

- **Step 1:** Add Gson Dependency:

```
xml

<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.8.8</version>
</dependency>
```

- **Step 2:** Create a Java Class: create a simple Java class called Person that we will use for encoding and decoding.

```
java

class Person {
    String name;
    int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```


Database and management system

Encoding is the process of converting a Java object into a JSON string. Here is how to do it:

java

Copy code

```
import com.google.gson.Gson;

public class EncodingExample {
    public static void main(String[] args) {
        // Create a Person object
        Person person = new Person("John", 25);

        // Create a Gson object
        Gson gson = new Gson();

        // Encode the Java object into a JSON string
        String json = gson.toJson(person);

        // Print the JSON string
        System.out.println("Encoded JSON: " + json);
    }
}
```

Output:

CSS

```
Encoded JSON: {"name":"John","age":25}
```

Decoding is the process of converting a JSON string back into a Java object. Here's how to decode JSON in Java:

java

Copy code

```
import com.google.gson.Gson;

public class DecodingExample {
    public static void main(String[] args) {
        // JSON string to decode
        String json = "{\"name\":\"John\",\"age\":25}";

        // Create a Gson object
        Gson gson = new Gson();

        // Decode the JSON string into a Person object
        Person person = gson.fromJson(json, Person.class);

        // Print the decoded Java object
        System.out.println("Decoded Person: Name = " + person.name + ", Age = " + person.age);
    }
}
```

Output:

yaml

Decoded Person: Name = John, Age = 25

❖ **JSON data types:** JSON (JavaScript Object Notation) is the most widely used data format for data interchange on the web. **JSON** is a lightweight text-based, data-interchange format and it is completely language-independent. JSON supports mainly 6 data types:

1. **JSON String:** JSON strings must be written in double quotes like C-language there are various special characters(Escape Characters) in JSON that you can use in strings such as \ (backslash), / (forward slash), b (backspace), n (new line), r (carriage return), t (horizontal tab), etc.

```
{ "name": "Vivek" }
{ "city": "Delhi\ / India" }
```

here \ / is used for Escape Character / (forward slash).

2. **JSON number:** A number in JSON can be an integer or a floating-point number. It does not have a specific type for integers or floats—both are treated as numbers.

```
{ "age": 20 }
{ "percentage": 82.44 }
```

3. **JSON Boolean:** A Boolean represents a true or false value. It can only have one of two values: true or false.

```
{ "result" : true }
```

4. **JSON Null:** The null type is used to represent an empty or unknown value. It is represented by the keyword null.

```
{
  "result" : true,
  "grade" : null,
  "rollno" : 210
}
```

5. **JSON object:** It is a set of name or value pairs inserted between {} (curly braces). The keys must be strings and should be unique and multiple key and value pairs are separated by a , (comma).

```
{
  "Geek":{ "name":"Peter", "age":20, "score": 50.05}
}
```

6. **JSON array:** It is an ordered collection of values and begins with [(left bracket) and ends with] (right bracket). The values of array are separated by , (comma).

```
{
  "geek":[ "Sahil", "Vivek", "Rahul" ]
}

{
  "collection" : [
    {"id" : 101},
    {"id" : 102},
    {"id" : 103}
  ]
}
```

❖ **Significance:**

1. JSON is human-readable and easy to write. The format uses simple text with key-value pairs (like a dictionary), which makes it intuitive for both developers and machines to understand.
2. Although JSON originated from JavaScript, it can be used with any programming language (such as Python, Java, C++, etc.). This makes it widely adopted across different platforms and technologies.
3. JSON has a lightweight structure with minimal syntax. It uses fewer characters compared to other formats like XML, which makes it faster to transmit over networks and easier to store.
4. JSON is primarily used for data exchange between a server and a web application. When an app needs to send data to a server, it can send it in JSON format. Similarly, the server can send data back to the app in JSON. This makes it very popular in web development, especially in APIs (Application Programming Interfaces).
5. JSON can be used on any platform (Windows, Mac, Linux) and any device (smartphones, computers, tablets), making it highly versatile.

❖ **XML:**

- **Xml** (Extensible Markup Language) is a mark up language. It is a way to store and transport data, similar to how a text document can store information.
- It is similar to HTML. That means, this scripting language contains various tags. But these tags are not predefined, in fact users can define their own tags.
- Thus HTML is designed for representing the data on the web page whereas the XML is designed for transport or to store data.

- **Significance/features:**

1. **Data Storage and Transport:** XML is mainly used to hold and send data between different systems or applications. It's not used to display data like a webpage does; instead, it's about carrying information from one place to another.
2. **Self-Describing:** XML is "self-describing," which means it contains both the data and the description of the data. For example, you can

have a piece of data like a name or age, and also define what that data represents, making it easier for others to understand the data.

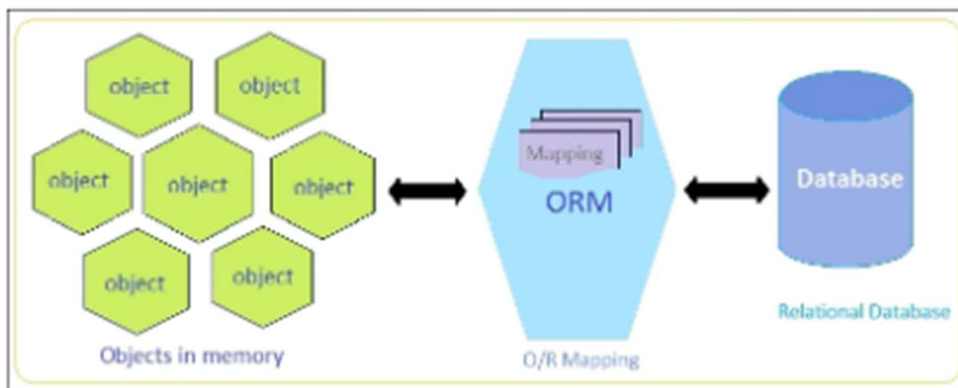
3. **Custom Tags:** Unlike HTML, where tags like `<h1>` or `<p>` are predefined, in XML you create your own tags to describe the data. For example, you might have a tag called `<name>` for a person's name, and `<age>` for their age.
4. **Not for Displaying Data:** XML is not about how data looks on a webpage. Instead, it's about how data is structured so that it can be easily read and understood by both machines and humans.
5. **Platform and Language Independent:** XML can be used on any computer or device, and it's not tied to any specific programming language. This makes it very flexible and useful for sharing data across different systems.

❖ **Uses of XML:**

1. When your data has multiple levels of hierarchy and complex relationships i.e. when your data naturally has a tree-like structure with parent and child relationships, like organizing products into categories and subcategories.
2. If you need to transfer data between different applications or platforms with varying data formats, converting the data to XML provides a common language for exchange.
3. XML's structure allows for easy modifications and additions to the data schema without significantly impacting existing applications
4. If your data keeps changing or different entries need different types of information, XML is flexible and allows you to store this kind of data easily.
5. XML is good for long-term storage because it is simple, text-based, and can be read by future systems. If you need to store data for a long time, XML is a reliable choice.
6. If you want your data to be easy to read and edit by people, XML is a good choice because it is stored in text format that is easy to understand.

❖ **Object-relational database system:**

- An Object-Relational Database System (ORDBMS) is a database system that combines features from both object-oriented databases and relational databases. It allows developers to use both relational data (tables, rows, columns) and object-oriented features (such as classes, inheritance, and polymorphism) within the same database system.
- The key idea behind an ORDBMS is to bridge the gap between relational and object-oriented systems, allowing developers to store complex data types (like objects) alongside traditional relational data.
- The concept of classes, objects and inheritance are supported in this database.
- **For example**, in an ORDBMS, you might have a table that stores customer information (like name and ID) and, in addition, a column that stores a more complex object, such as an address with fields for street, city, and zip code.



- **Advantages:**

- Supports Complex Data Types: They can be used to store objects, collections and user-created data types.
- It supports features of object-oriented programming such as inheritance and polymorphism.

- **Disadvantages:**

- Because ORDBMS contains aspects of both relational and object-oriented databases, it can be more difficult to manage that type of database than a relational database.

- It may add further overhead when dealing with objects as opposed to the easier relational model.
- Currently, there are not many well-established SQL standards for object-relational systems as compared to stand alone Relational DB's.

❖ Relational vs object-relational:

RDBMS	ORDBMS
RDBMS is a Relational Database Management System based on the Relational model of data.	ORDBMS is a Object Oriented Relational Database Management System based on the Relational as well as Object Oriented database model.
It follows table structure, it is simple to use and easy to understand.	It is same as RDBMS but it has some extra confusing extensions because of the Object Oriented concepts.
It has no extensibility and content.	It is only limited to the new data-types.
Since RDBMS is old so, it is very mature.	It is developing so it is immature in nature.
In this, there is extensive supply of tools and trained developers.	It can take the advances of RDBMS tools and developers.
It has poor support for Object-Oriented programming.	It supports the features of object-oriented programming.
It supports Structured Query Language (SQL).	It supports Object Query Language (OQL).
RDMS is used for traditional applications tasks such as data administration and data processing.	ORDMS is used for applications with complex objects.
It is capable of handling only simple data.	It is also capable of handling the complex data.
MS SQL server, MySQL, SQLite, MariaDB are examples of RDBMS.	PostgreSQL is an example of ORDBMS.

❖ **Table inheritance:**

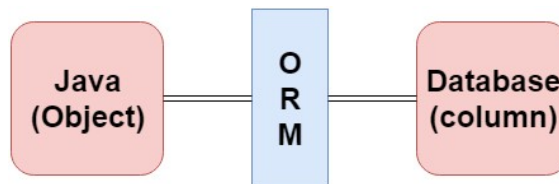
- Table inheritance in a Database Management System (DBMS) refers to the concept where one table can inherit attributes from another, forming a hierarchical relationship between tables.
- This concept is borrowed from object-oriented programming, where a subclass can inherit attributes and behaviors from a superclass.
- In the context of databases, table inheritance allows a table to inherit columns (attributes) from another table while still maintaining its own unique columns.
 1. **Single table inheritance:** Single Table Inheritance, also known as Table per Hierarchy (TPH), stores all entities like **superclass** and **subclass** in a single table. **Example:** If you have a Person table and two types of people: Employee and Customer, you could store all data in one Person table. The Employee specific attributes like employee would be null for Customer records, and vice versa.
 2. **Class Table Inheritance (CTI):** In this model, a separate table is created for each class (parent and child), and the child table will include a foreign key reference to the parent table. **Example:** In a Person, Employee, and Customer schema, the Employee and Customer tables will each store their own specific data and will refer to the Person table via a foreign key.
 3. **Concrete Table Inheritance (CTI):** Each table (parent and child) has its own copy of the inherited attributes. There's no common parent table, but instead, each child table contains the full set of attributes for that class, including the ones inherited from the parent class, leading to possible redundancy. **Example: Employee:** Contains common attributes like name, address, and employee_id. **FullTimeEmployee:** Inherits name, address, and employee_id, and adds salary and benefits. **PartTimeEmployee:** Inherits name, address, and employee_id, and adds hourly_rate and hours_worked.

❖ **Object-Relational mapping:**

- Object Relational Mapping (ORM) is a functionality which is used to develop and maintain a relationship between an object and relational database by

mapping an object state to database column. It is capable to handle various database operations easily such as inserting, updating, deleting etc.

- Object-Relational Mapping (ORM) is a programming technique that allows developers to interact with relational databases using object-oriented programming languages.
- Instead of writing raw SQL queries to interact with the database, ORM enables the use of objects and classes in the programming language to perform database operations like insert, update, delete, and retrieve.
- ORM frameworks are: Hibernate, Toplink, ORMLite, iBATIS, JPOX



- Imagine you are working with a Python application that needs to interact with a SQL database. Without ORM, you would have to write SQL queries to interact with the database. However, with ORM, you define Python classes that map to database tables.

For example, if you have a table called `Person` in a database and you want to fetch data from it, you would typically write a SQL query like:

sql

Copy code

```
SELECT * FROM Person WHERE age > 30;
```

With ORM, you create a Python class, say `Person`, that represents this table. Instead of writing SQL, you can interact with the data like this:

python

Copy code

```
# Assuming you are using an ORM library like SQLAlchemy
person = session.query(Person).filter(Person.age > 30).all()
```

🗺️ **Spatial data:** Spatial databases are specialized databases designed to store and manage spatial data, which includes geographic or geometric information associated with objects in space. These databases are commonly used in GIS (Geographic Information Systems), location-based services, and various applications involving spatial analysis. E.g. A **spatial database** is often used in GIS applications to manage geographic data such as roads, rivers, parks, and buildings. These systems rely on spatial queries to perform tasks like: Mapping, Route planning, Environmental monitoring etc.

❖ **Geometric data:**

- **Geometric data** refers to data that represents shapes and structures in space. This data can describe objects like points, curves, lines, and polygons, often used for designing physical or abstract objects in 2D or 3D space.
- **Example:** A rectangle's shape can be defined by its corners (coordinates of four points), or a road can be represented by a line.
 - **Use Case:** Used in areas like engineering, architecture, and computer graphics to describe the form and structure of objects.

❖ **Geographic data:**

- **Geographic data** refers to information about locations on Earth, typically represented by coordinates such as **latitude** and **longitude**. It describes positions or features in the real world, often on a global scale. Geographic data is commonly used in applications like maps, navigation systems, and GPS devices.
- **Example:** A city's location represented by its latitude (40.7128° N) and longitude (74.0060° W) coordinates.
 - **Use Case:** Mapping locations, tracking movement, or providing directions.
-